Virtual Gold x CMU (Hybrid Architecture)

Capstone Midterm Presentation | October 5, 2026

CMU Capstone Team

Zhexuan Ye, Anmol Sahu, Yudi Guo, Karina Yang, Mark Chu

Heinz College of Information Systems and Public Policy

Carnegie Mellon University

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Five team members each own one part of the service

1

Zhexuan Ye

Service engineering

2

Anmol Sahu

Model work

3

Yudi Guo

Privacy research

4

Karina Yang

Evaluation and test data

5

Mark Chu

Integration and coordination

Faculty advisor: Stacy Rosenberg

Client: Virtual Gold Inc.

One email shows the decisions the service must make



Figure 1. Synthetic document example. Private spans become placeholders; the public support contact stays visible.

Local privacy screening supports hybrid AI use

Project objective

Develop and evaluate a local privacy service for English-language text and documents, with controlled cloud release and local restoration.

Presentation focus

Share our research, service architecture, evaluation approach, and next steps.

Success combines privacy protection, useful output, and reliable service performance.

English-language text and documents are the service inputs

1

Inputs

- English document text
- Task or purpose
- Source context
- Email benchmark: 100 fictional examples

2

Service deliverables

- VG08-integrated service
- Versioned API contract
- Tier + routing rules
- Monitoring dashboard

3

Evaluation + handoff

- Labelled benchmark
- Evaluation + ablations
- Final presentation
- Documentation + academic report

Agenda

1. Sensitive Data Classification
   Yudi Guo
2. Architecture Design
   Zhexuan Ye, Anmol Sahu
3. Service Engineering Plan
   Zhexuan Ye
4. Evaluation Dataset
   Karina Yang
5. Summary and Next Steps
   Mark Chu

Sensitive Data Classification

Yudi Guo

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Six client requirements and how the proposal meets each

Requirement	How the proposal meets it
Strip private data before the cloud	Redact with reversible placeholders and restore the reply
Run locally on an open-source model	Run on own hardware with no outbound network access
Work automatically on every request	Decide in code on every request, with no per-item review
Judge context, not just patterns	Read context, such as “my cell” vs. “our support line”
Output a confidence score	Calibrate the model score on labeled emails
Measure privacy and usefulness	Evaluate leakage, utility, and service performance

Stated main challenge: determining what counts as private.

Table 1. Client requirements and how the proposal meets them.

Research 1 of 3: detection, confidence, and contextual privacy

Research	Finding / focus	Design implication
Rescriber (CHI 2025)	Local-model redaction can miss private spans	Combine candidate detectors
Xiong et al. (ICLR 2024)	Expressed confidence is often miscalibrated	Calibrate scores on held-out labels
RedactionBench (Jun 2026)*	Privacy depends on document context	Separate mandatory and contextual labels
DR-SL (Sep 2026)*	Verifies privacy and task-information preservation	Evaluate release and restored utility

Research 2 of 3: privacy beyond entity detection

Paper (2026 preprint)	Contribution	Capstone application
RAT-Bench (Feb) arXiv:2602.12806	Anonymization evaluation includes re-identification risk	Treat detector recall as one measure, not proof of privacy
LLM-Redactor (Apr) arXiv:2604.12064	Compares eight request-privacy techniques	Compare redaction and local routing; measure restored utility
MINIM (Jun) arXiv:2606.13949	Trusted local sanitization produces a task-relevant view	Evaluate information needed for the task as well as sensitivity

Research 3 of 3: context and confidence validation

Paper	Finding / focus	Evaluation implication
Semantic Entropy Probes (Kossen et al., 2024)	Predicts semantic uncertainty using hidden states	Compare confidence signals where local runtime supports them
MuPPET (Jun 2026)*	Multi-party privacy requires recipient tracking	Include recipient/purpose context in adversarial examples
PrivacyAlign (Jun 2026)*	Human annotations ground privacy judgments	Use label explanations and adjudicate ambiguous examples

The category sets how severe a leak would be.
The context sets how confident we are.

When confidence is insufficient, the item is redacted by default and replaced with a reversible placeholder, so the assistant can still do its job.

The model judges – code decides

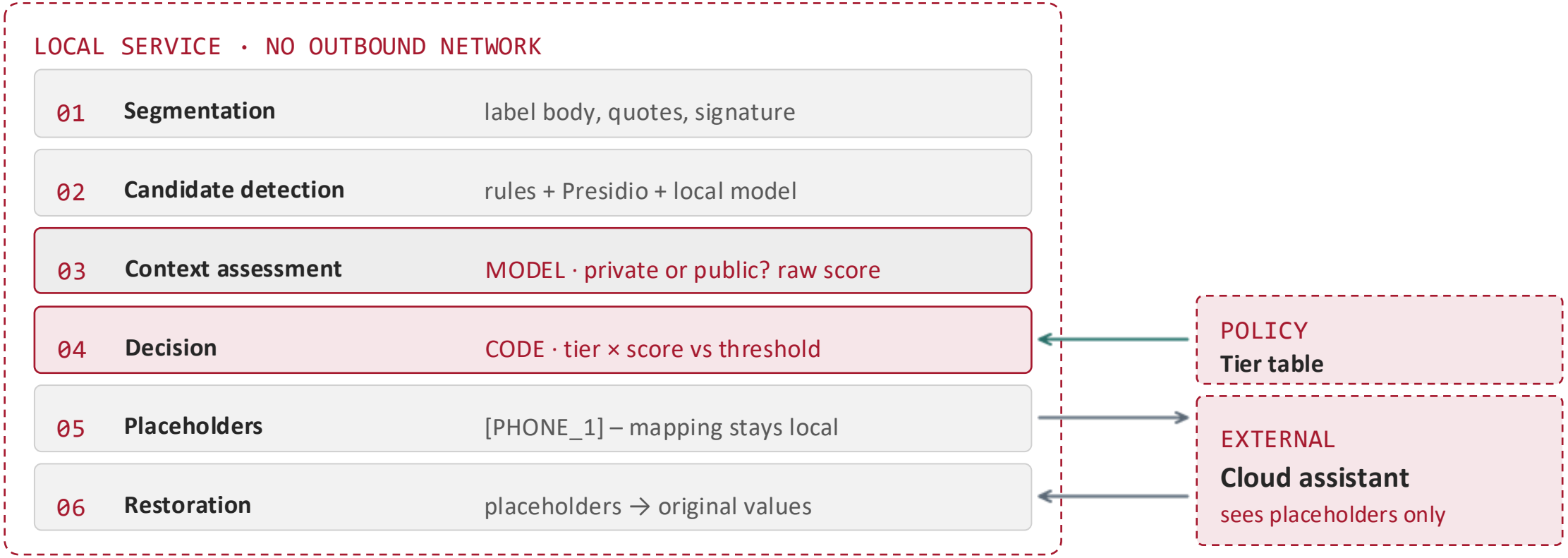


Figure 2. Decision steps inside the local service. Source: Project team.

More harm, less proof required

Tier	Categories	Redaction rule
High	IDs, passwords and keys, bank and card numbers, health, business-confidential	Redact at score ≥ 0.2
Medium	Names of private individuals, personal phone, personal email, home address	Redact at score ≥ 0.5
Low	City, dates, organisation names, age, gender	Keep alone – redact if combined

Table 3. Sensitivity tiers and redaction rules. Note. Tiers follow NIST SP 800-122 (McCallister et al., 2010). Thresholds are starting values.

Two phone numbers, two outcomes

INCOMING EMAIL

Hi Tom,

Before Thursday's board call: my **oncologist** moved my appointment, so I'll dial in from home. Reach me on my cell, **412-555-0182**. On the vendor outage, Acme's support line is **1-800-555-0199**.

— Dana

Item	Context	Score	Decision
Dana, Tom	private individuals	0.90	[PERSON_n]
oncologist	"my appointment"	0.95	[HEALTH_1]
412-555-0182	"my cell"	0.95	[PHONE_1]
1-800-555-0199	toll-free, "support line"	0.15	Keep
Acme, Thursday	Low tier, alone	n/a	Keep

Table 4. Items in the example email and the resulting decisions. Note. Synthetic example, scores are illustrative.

What the cloud assistant receives

SENT TO THE CLOUD

Hi **[PERSON_2]**,

Before Thursday's board call: my **[HEALTH_1]** moved my appointment, so I'll dial in from home. Reach me on my cell, **[PHONE_1]**.

On the vendor outage, Acme's support line is 1-800-555-0199.— **[PERSON_1]**

Still possible

Draft the reply to [PERSON_1]. Summarise the Acme outage.

Never leaves the machine

The diagnosis, the personal number, both names.

On the way back

Placeholders are restored locally before the executive reads the draft.

Calibrating the model's score

Language models tend to be overconfident about their own answers (Xiong et al., 2024). The raw score is therefore corrected before any threshold is applied.

1. Run the model on labelled emails
2. Group items by raw score
3. Measure how many are actually private
4. Use that rate as the calibrated score

Raw score	Actually private	Calibrated
0.9 – 1.0	140 of 200	0.70
0.7 – 0.9	50 of 100	0.50
0.5 – 0.7	15 of 50	0.30
0.0 – 0.5	15 of 150	0.10

Table 5. Example calibration of raw model scores. Note. Illustrative values: a raw 0.95 would be reported as 0.70.

One third of personal data missed

0.63

Recall of a local Llama3-8B in the Rescriber study (Zhou et al., 2025): about one in three personal items missed. GPT-4o reached 0.88 and Presidio 0.68.

Rescriber's users reviewed every message. Here, a miss goes straight to the cloud. After measuring our own baseline on email, we add one measure at a time:

- Combine all three detectors
- Lower tier thresholds
- Short email examples in the prompt
- …

Leak rate per tier is the primary evaluation metric

DATA

100 emails

Every private item labelled by category, and private or public in context

PRIMARY METRIC

Leak rate

Share of private items that reach the cloud, per tier

ALSO TRACKED

Utility

Over-redaction, calibration, seconds per email

Ablation: each step is added to a Rescriber-style baseline, tuned and tested on separate subsets

Baseline | + detector union | + name propagation | + repeated runs | + tier thresholds

The service combines existing components with two new contributions

Component	Effort
Rules and Presidio	Low
Local model via Ollama, using Rescriber's open-source prompt	Low to medium
Merging the three detectors' outputs	About 50–100 lines

Table 6. Existing components and estimated integration effort. Note. Speed depends on hardware. Zhou et al. (2025) report a 3.4 s mean for Llama3-8B on an M2 laptop.

Architecture Design

Zhexuan Ye, Anmol Sahu

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Three principles shape every design decision

1

Local by default

All detection and judgment run on project hardware, with no outbound network access.

2

Model proposes, code decides

The model finds and scores spans – tier rules in code make every redaction decision.

3

Fail safe

When a component fails or an item cannot be classified, the service redacts.

One local model sits inside a pipeline of code and data stores

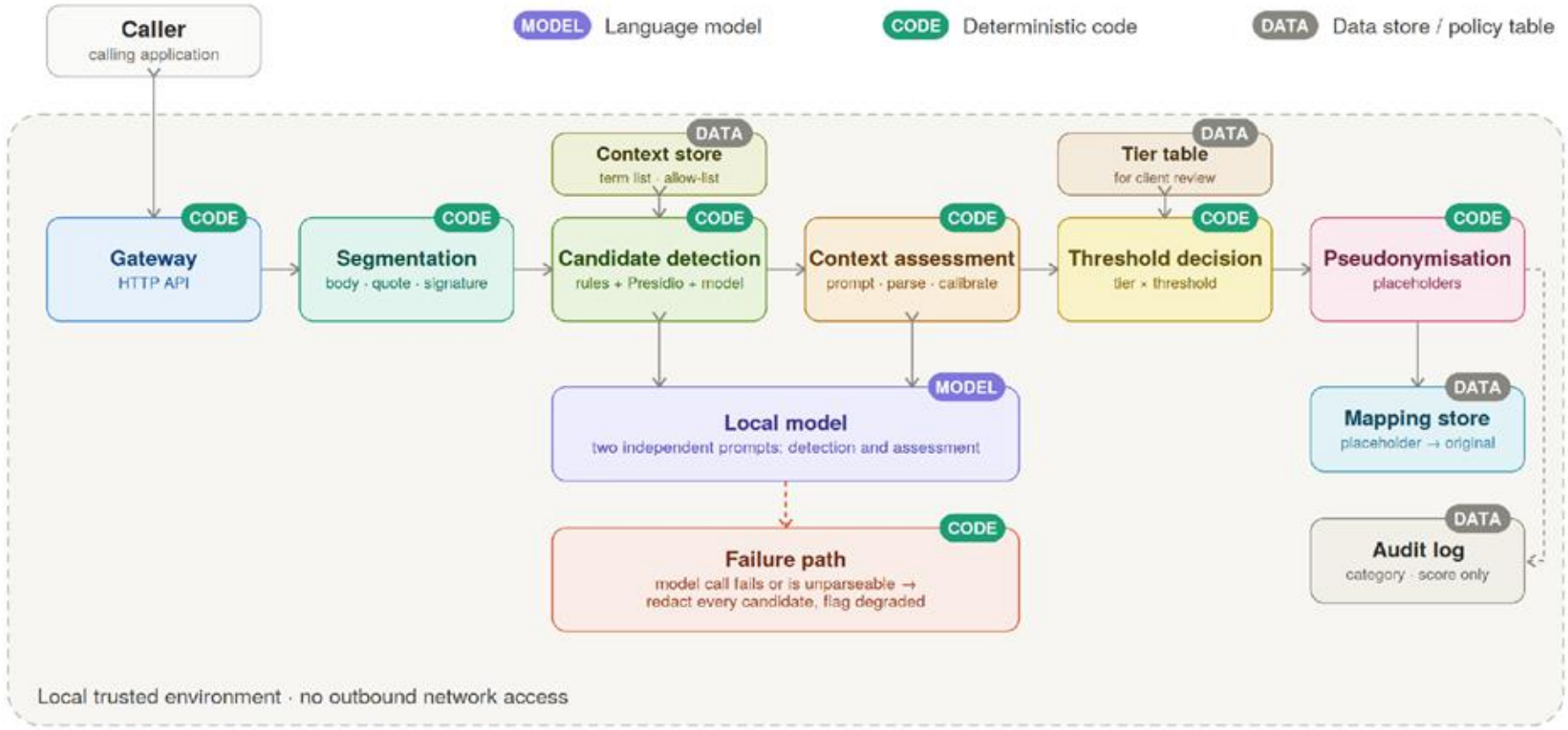
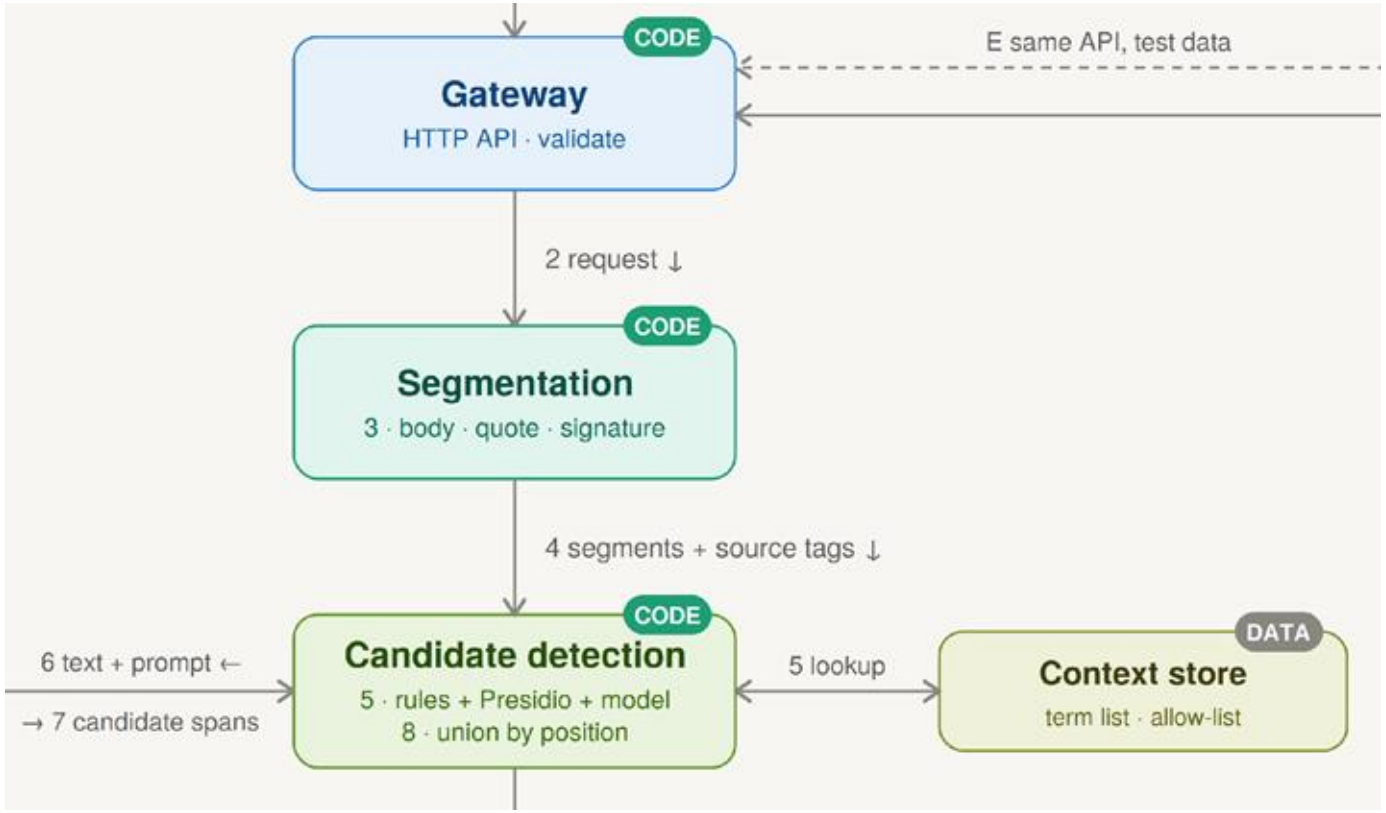


Figure 3. Component map of the local redaction service. Source: Project team.

Segmentation and three-way detection turn raw text into candidate spans



3 Split into body, quote and signature

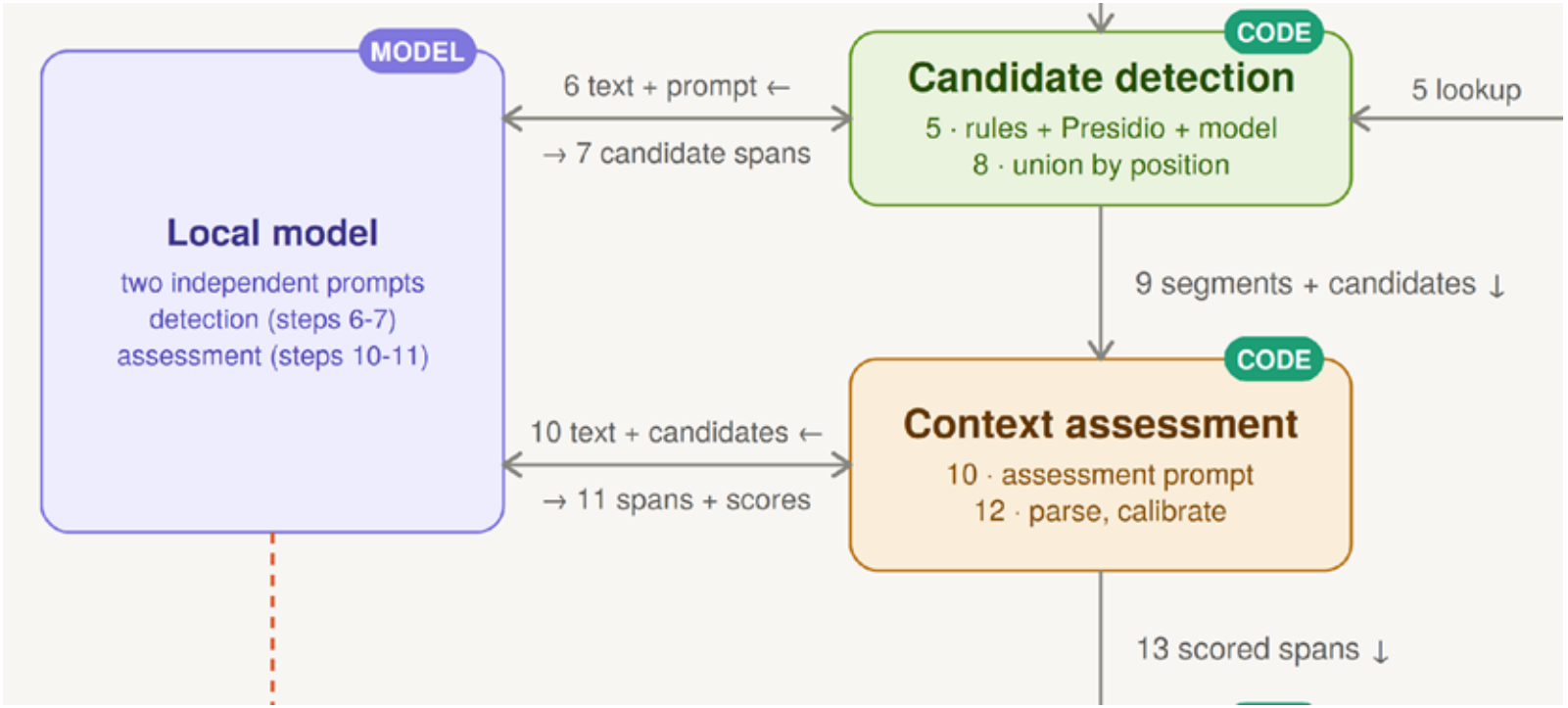
4 Pass segments with source tags

5 Run rules and Presidio – check the context store

6–8 Add model candidates – union all three sources

Figure 4. Input side: gateway, segmentation and candidate detection (rules, Presidio and model).

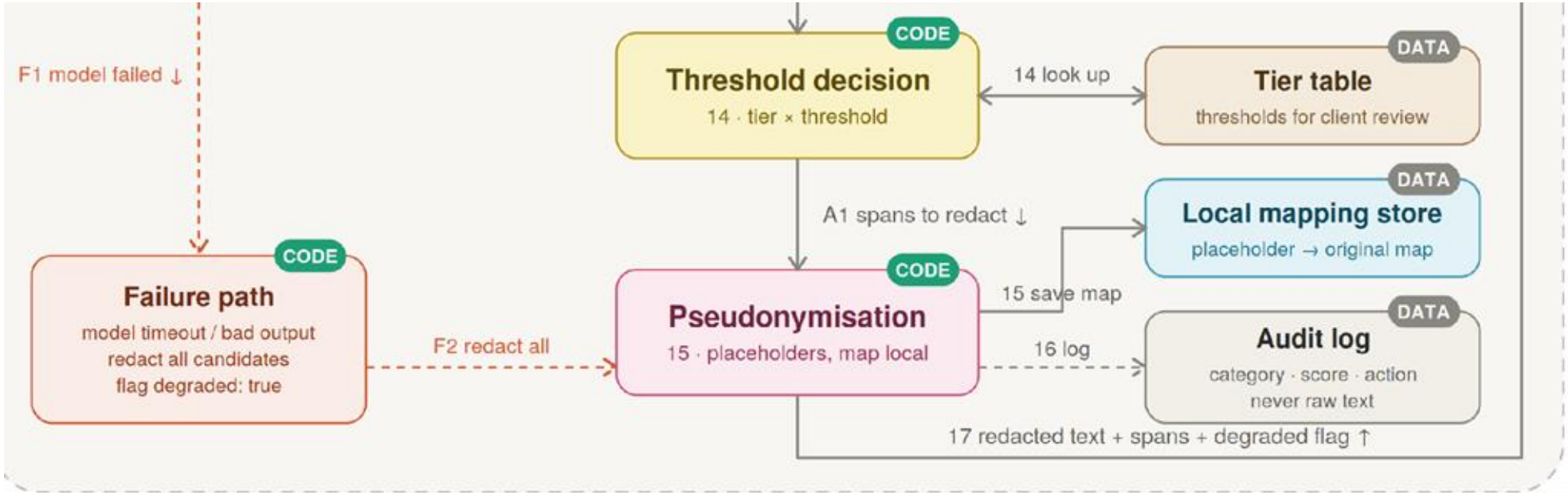
One local model serves detection and assessment with independent prompts



Detection prompt: steps 6–7. Assessment prompt: steps 10–11.

Figure 5. One local model, called with two independent prompts. Source: Project team.

Code applies the tiers, then replaces spans with placeholders



14 Tier table sets the threshold for each span

15 Placeholders replace spans – the map stays local

16 Audit log records category and score, never raw text

Figure 6. Decision, pseudonymisation, mapping store, audit log and failure path. Source: Project team.

Steps 1–5: the request is split and rule-based candidates are found

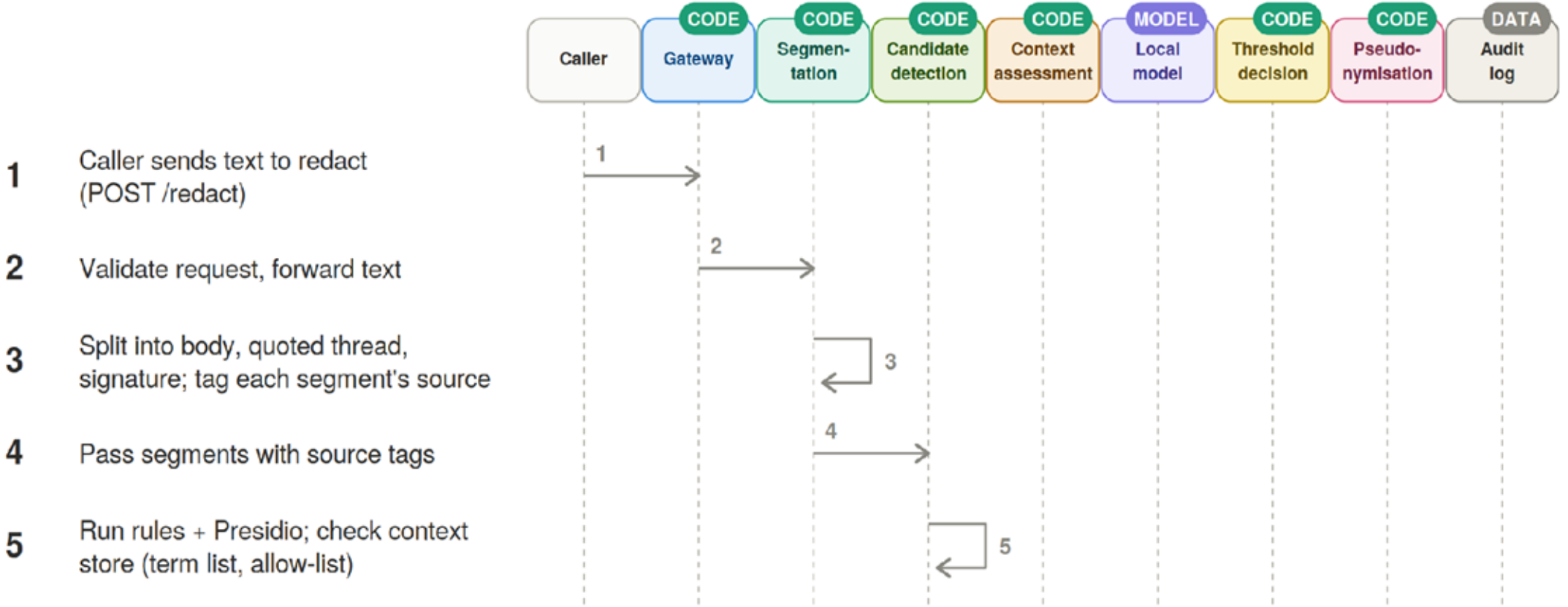


Figure 7. Request workflow, steps 1–5. Source: Project team.

Steps 6–11: the model is called twice, with two independent prompts

6	Send text + detection prompt (in parallel with step 5)
7	Return candidate spans
8	Union the three sources by character position
9	Pass segments + candidates to assessment
10	Send text + candidates with the assessment prompt
11	Return private? + raw score per span

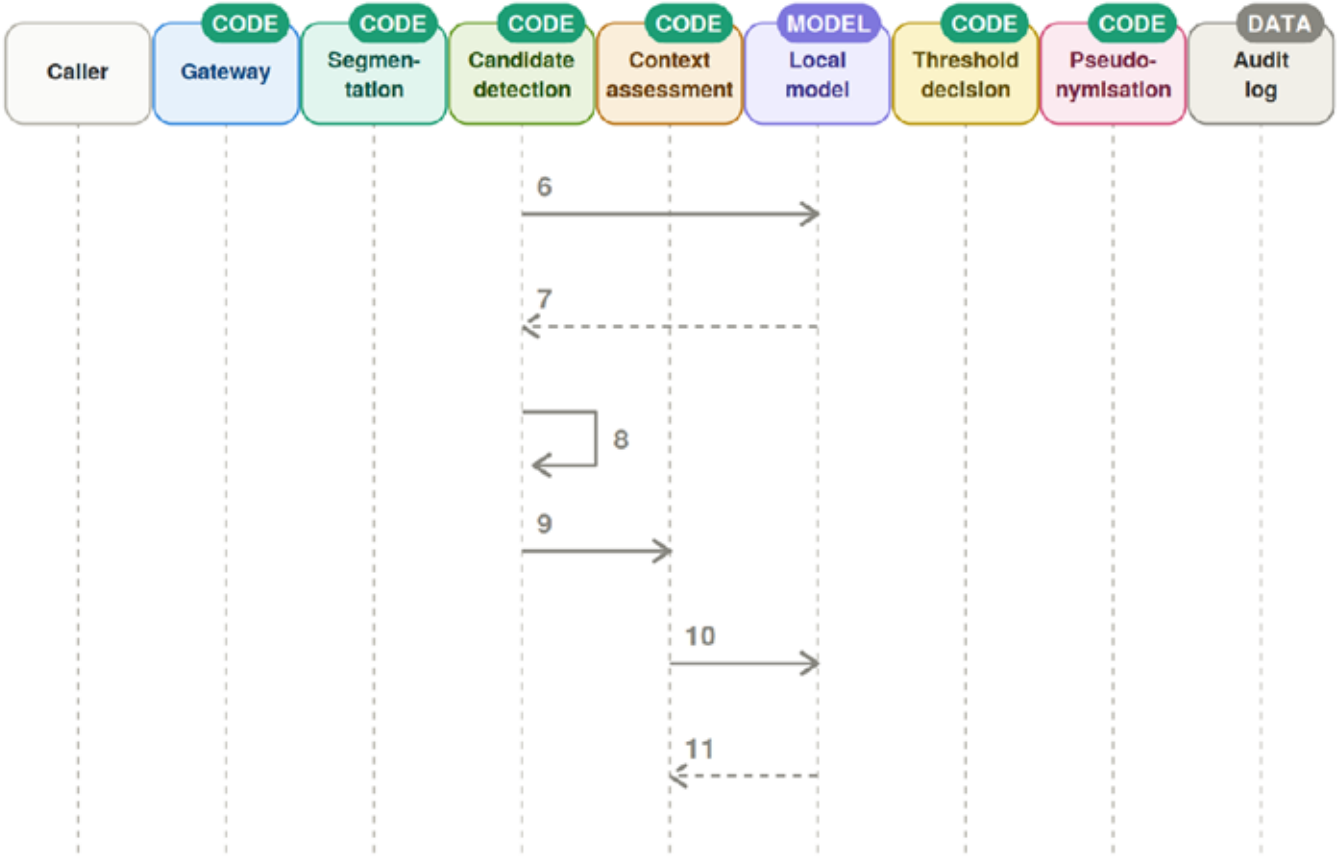


Figure 8. Request workflow, steps 6–11. Source: Project team.

Steps 12–14: code turns scores into decisions – only qualifying spans are replaced

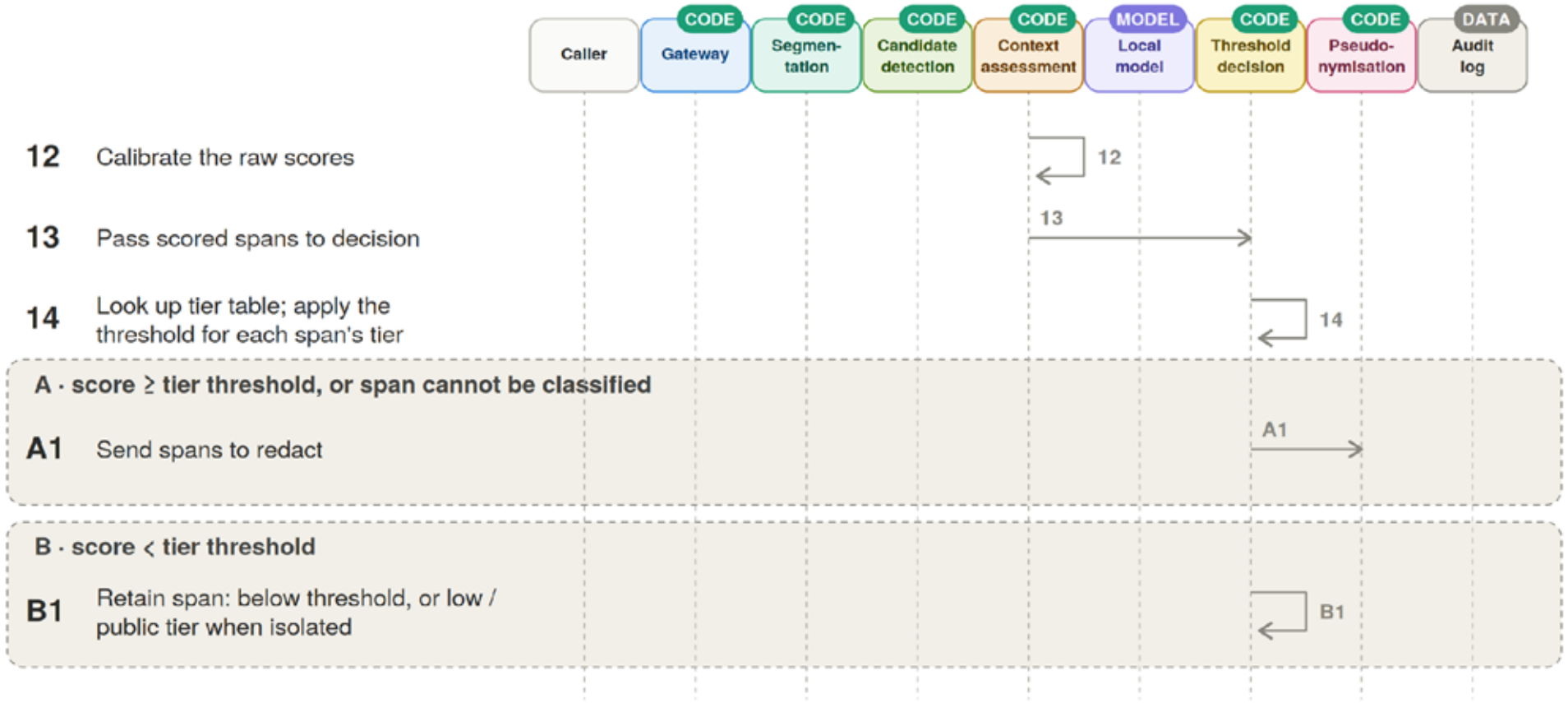


Figure 9. Request workflow, steps 12–14 and decision outcomes A and B. Both branches continue to step 15. Source: Project team.

Steps 15–18: redacted text returns and the decision is logged

15 Replace redacted spans with placeholders; save mapping locally

16 Log category, score, action, degraded flag (never raw text)

17 Return redacted text + spans (+ degraded flag)

18 Deliver redacted text to caller

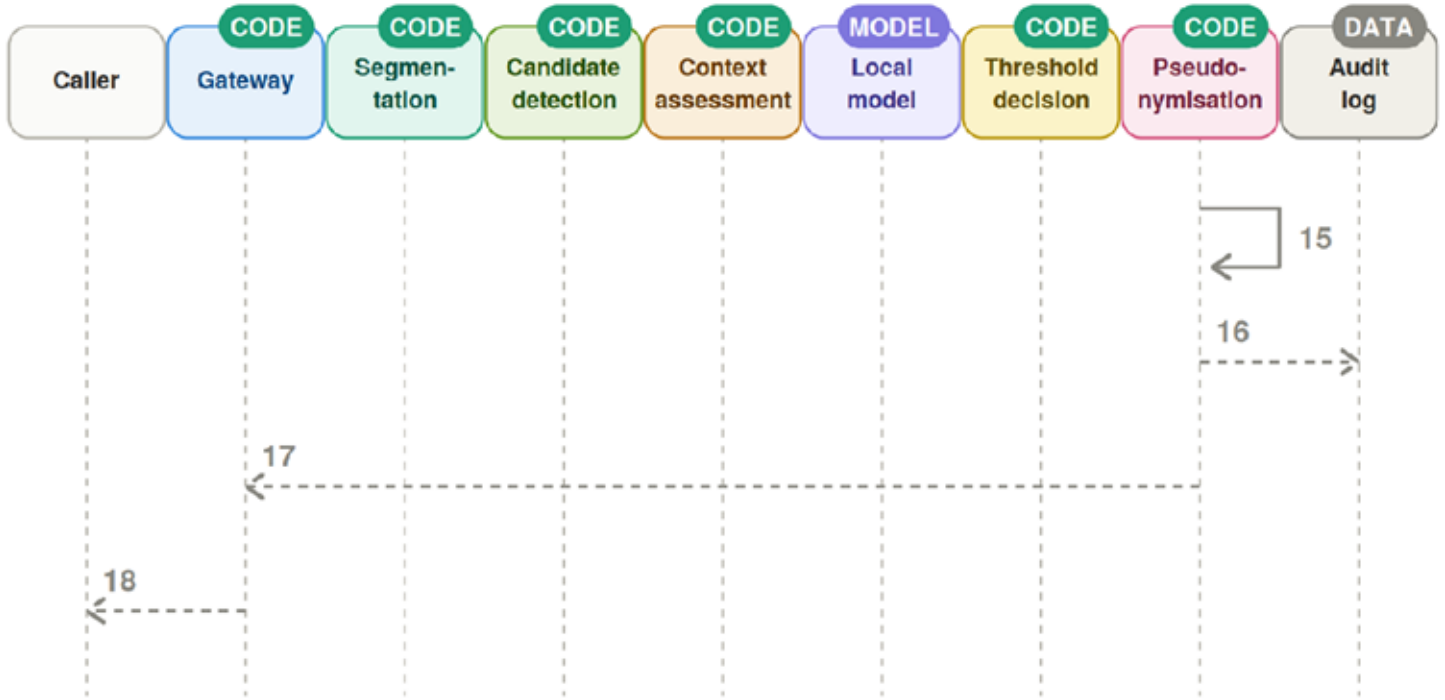


Figure 10. Request workflow, steps 15–18. Source: Project team.

Restore runs later as a separate, deliberate call

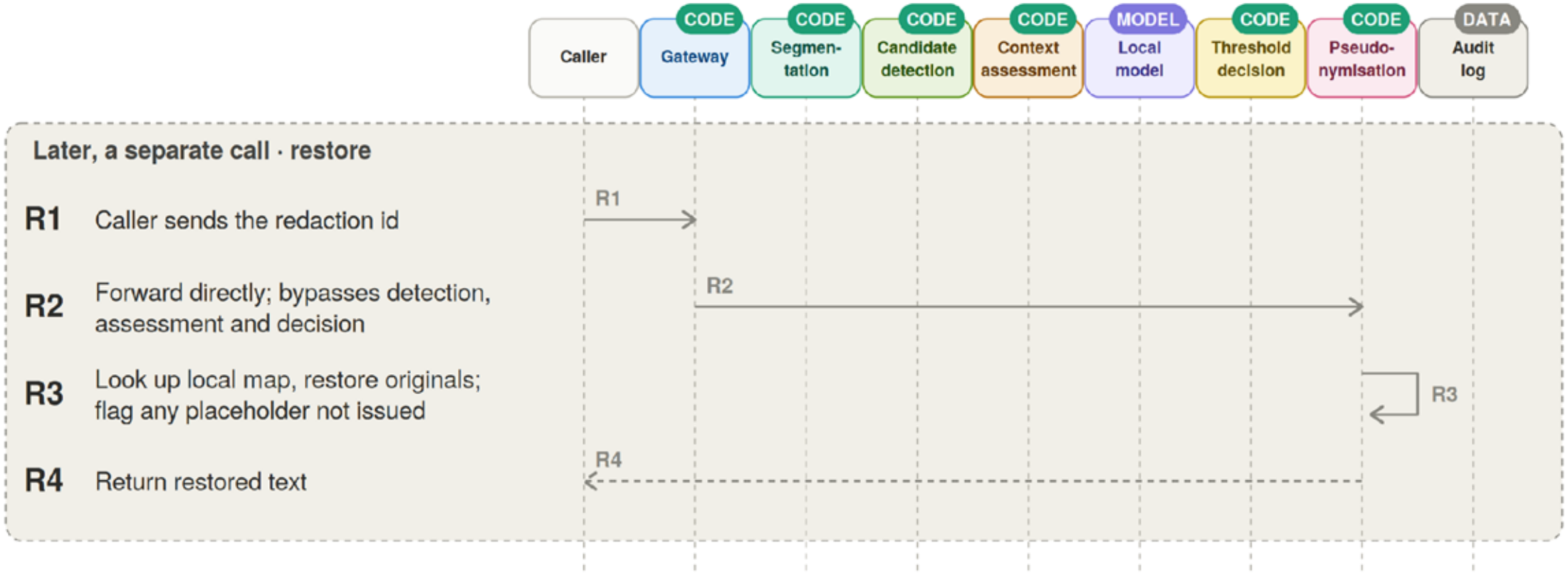
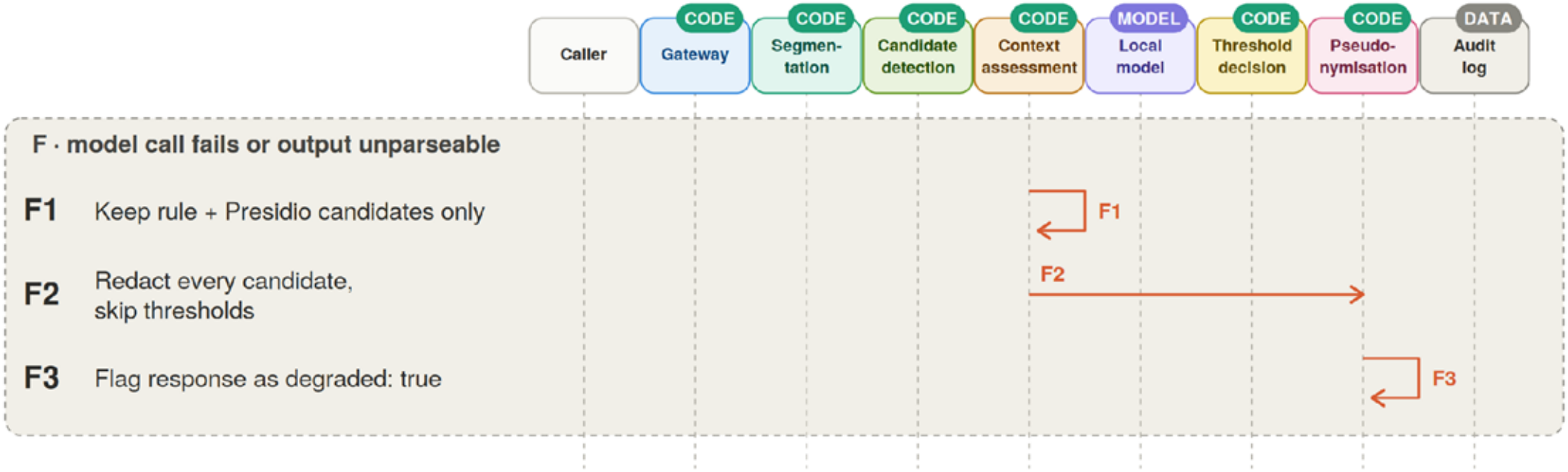


Figure 11. Restore request, handled separately from redaction. Source: Project team.

Any failure redacts every candidate and flags the response



Also flagged: any placeholder in a reply that the service did not issue.

Figure 12. Failure path when the model is unavailable or its output cannot be parsed. Source: Project team.

VG08 sends only verified, sanitized content

Local privacy service

Screen English text + task

Redact and apply routing

Return status + degraded flag

VG08 host

Check release eligibility

Use exact sanitized fields

VG08 owns provider calls

Cloud assistant

Receives retained text and placeholders

Returns an answer

Local restoration uses issued tokens; the placeholder map stays within the service.

Figure 8. Local privacy service and VG08 boundary.

Service Engineering Plan

Zhexuan Ye

Carnegie Mellon University
HeinzCollege
INFORMATION SYSTEMS • PUBLIC POLICY • MANAGEMENT

Service engineering owns the gateway and the wiring of the whole service

Final engineering deliverables

- Authenticated gateway and request validation
- Pipeline wiring, routing, and degraded fallback
- Local records: restore, delete, and lifecycle controls
- Metrics and logs, never raw text
- Dashboard: Prometheus and Grafana

A live dashboard will show the health of the service

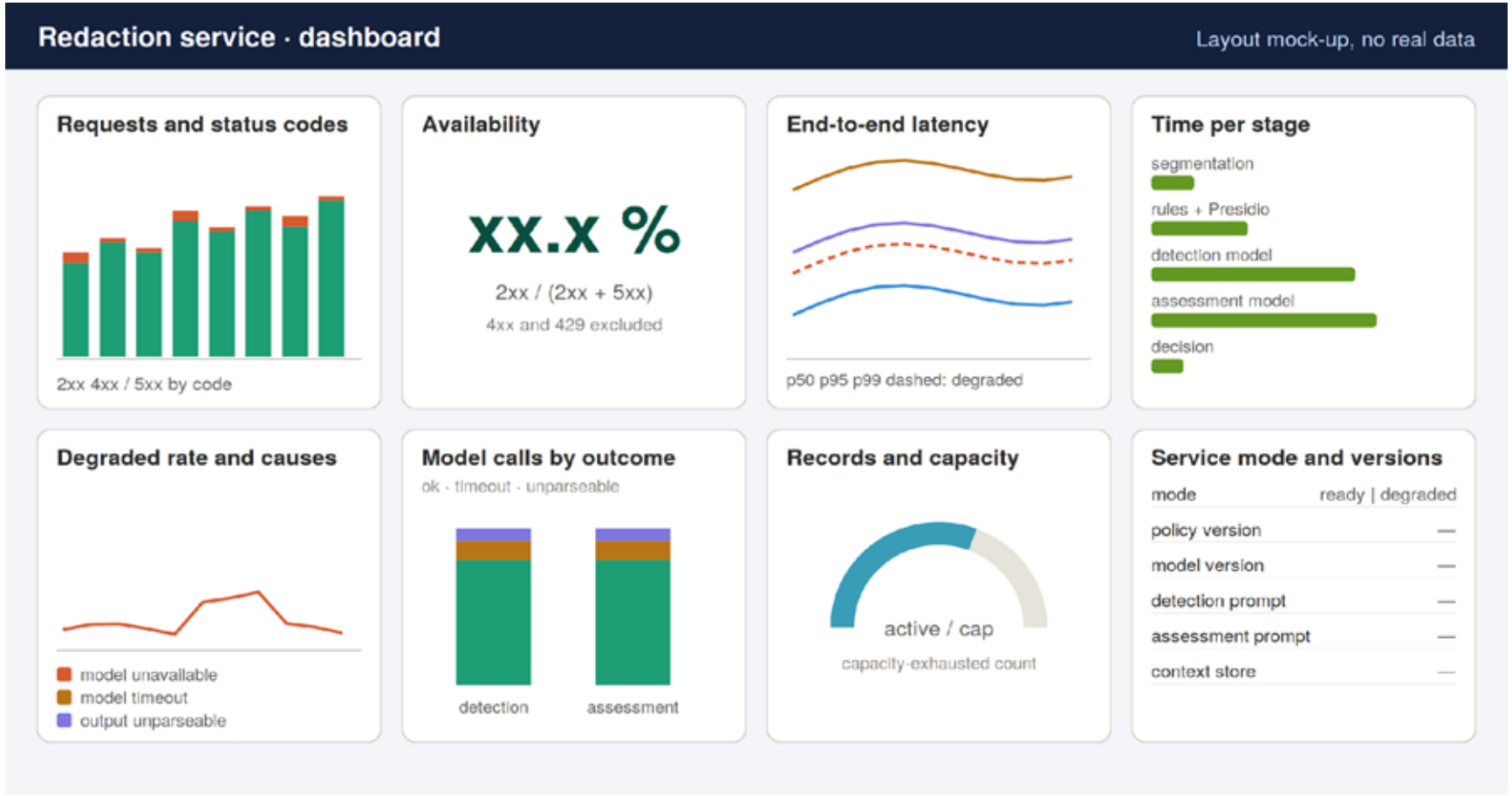


Figure 13. Planned dashboard layout (mock-up, no real data).

The dashboard tracks service health, readiness, and release outcomes

Service health

- Request count and errors
- End-to-end latency
- Per-stage timings

Processing readiness

- Model / recognizer state
- Degraded rate and causes
- Model-call outcomes

Release outcomes

- Routing status counts
- Record capacity
- Policy, model, prompt versions

Benchmark leak rate is evaluated with labels; live request status is not proof of accuracy.

Figure 7. Dashboard metrics. Source: Project team.

Evaluation Dataset

Karina Yang

The dataset holds 100 fictional emails with 999 labeled items

100

realistic executive emails, all fictional

15

business areas, from finance to personal

9

industries and 4 exec roles

999

labeled sensitive items, 18 types

TEST CASES BUILT IN

Context cases

Personal cell next to a public support line

10 clean emails

Nothing sensitive, to measure over-masking

Forwarded threads

Names quoted inside reply chains

5 attack emails

Hidden prompt injection, spelled-out numbers

56 business secrets

Sensitive sentences with no single name

Each email is labeled by business context and by sensitive item

EMAIL E001 · FINANCE · CFO

From: **Megan Ostrowski** PERSON

Victor PERSON, the warranty reserve needs to go up by **$1.84M** AMOUNT because of the pump returns.

Office: **(617) 555-0142** PHONE

Task: "Summarize this and tell me what I need to approve."

Secrets with no single name, like "we're closing the Fresno site", get a **FACT** label.

Per email	Example
Business area	Finance & Accounting
Industry	Medical devices
Exec role	CFO
Task	Summarize, flag approvals

Per label	Meaning
Type	One of 18: person, phone, amount, codename…
Position	Exact start and end character
must_mask	true = leak if sent raw – false = public, fine either way

Table 7 (top). Fields recorded for each email. Table 8 (bottom). Fields recorded for each labeled item.

One JSON line per email links the dataset to model output

DATASET RECORD · ONE LINE PER EMAIL

```
{
  "id": "E001",
  "domain": "Finance & Accounting",
  "task": "Summarize this and…",
  "text": "From: Megan Ostrowski…",
  "spans": [
    {"start": 6, "end": 21,
     "type": "PERSON",
     "must_mask": true}
  ]
}
```

Also records **industry**, **exec** role and **tags**. **spans** is the answer key: every sensitive item and its exact position.

MODEL OUTPUT · ONE LINE PER EMAIL

```
{
  "id": "E001",
  "spans": [
    {"start": 6, "end": 21,
     "type": "PERSON",
     "score": 0.93}
  ]
}
```

The model reads only **text** and **task**, never the answer key. It returns what it masked: **start** and **end** are required, **type** and confidence **score** are optional.

Summary and Next Steps

Summary: one local service, four workstreams

1

Sensitive data classification

- Category sets severity, context sets confidence
- Calibrated confidence scores
- Main risk: missed personal data

2

Architecture design

- Everything runs locally
- The model proposes, code decides
- Any failure redacts every candidate

3

Service engineering

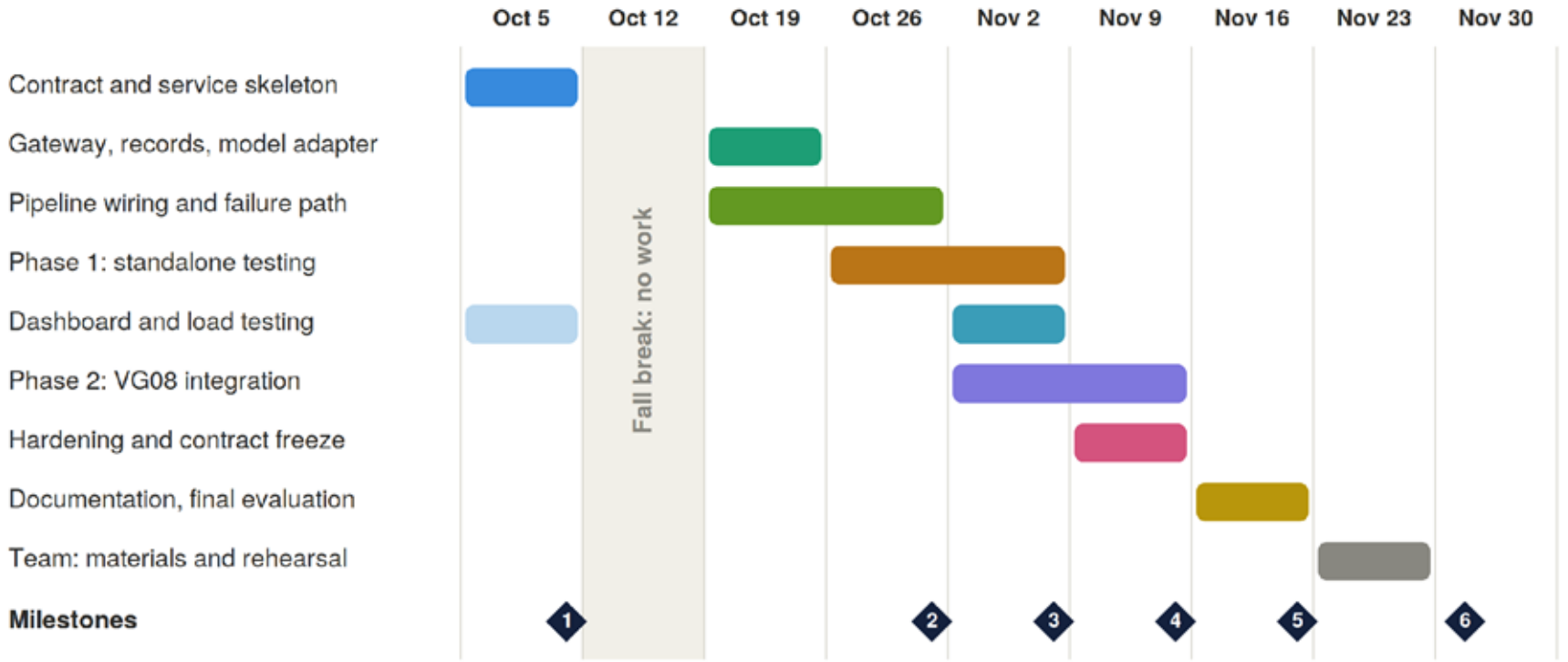
- Gateway, pipeline, and request records
- Six milestones to November 30
- Live Prometheus and Grafana dashboard

4

Evaluation dataset

- 100 fictional executive emails
- 999 labeled items, 18 types
- One shared format to score every model

Build and test the service through six delivery milestones



1. Oct 11 · Contract 0.4.2 and service skeleton
2. Nov 1 · End-to-end pipeline with failure path
3. Nov 8 · Dashboard and load-test results
4. Nov 15 · VG08 integration; contract freeze
5. Nov 22 · Code freeze (soft freeze Nov 18)
6. Nov 30 · Final presentation

Figure 14. Project Timeline, October 5 to November 30.

Standalone testing precedes VG08 integration

Phase 1: October 26–November 8

- Test English text and documents
- Check redaction + restoration
- Measure benchmark performance
- Exercise degraded fallback

Phase 2: November 2–15

- Integrate with the VG08 sandbox
- Validate complete workflows
- Check routing before cloud use
- Freeze the API contract

Thank you. Questions?

We welcome questions and feedback on the midterm progress.

Team member	Workstream	Contact
Zhexuan Ye	Service engineering	zhexuany@andrew.cmu.edu
Anmol Sahu	Model evaluation	anmolsah@andrew.cmu.edu
Yudi Guo	Privacy research	yudig@andrew.cmu.edu
Karina Yang	Evaluation and test data	karinay@andrew.cmu.edu
Mark Chu	Integration and coordination	markchu@andrew.cmu.edu

References

Carnegie Mellon University. (2026). *Midsemester presentation guidance* [Course handout].

CMU Capstone Team. (2026). September 28 Virtual Gold weekly sync [Client meeting notes].

CMU Capstone Team. (2026b). *Virtual Gold midterm evaluation dataset inventory* [Unpublished presentation inventory].

McCallister, E., Grance, T., & Scarfone, K. (2010). *Guide to protecting the confidentiality of personally identifiable information (PII)* (NIST Special Publication 800-122). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-122

Microsoft. (n.d.). *Presidio: Data protection and de-identification SDK*. https://microsoft.github.io/presidio/

References

Virtual Gold Inc. (n.d.). *Virtual Gold Inc. - AI assistant* [Client project brief].

Xiong, M., Hu, Z., Lu, X., Li, Y., Fu, J., He, J., & Hooi, B. (2024). Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *The Twelfth International Conference on Learning Representations.*

https://proceedings.iclr.cc/paper_files/paper/2024/hash/6733cf15e10e2cd1d59af033c3bb8507-Abstract-Conference.html

Zhou, J., Xu, E., Wu, Y., & Li, T. (2025). Rescriber: Smaller-LLM-powered user-led data minimization for LLM-based chatbots. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems* (pp. 1–28). Association for Computing Machinery.

https://doi.org/10.1145/3706598.3713701

References: recent privacy research

Krčo et al. (2026). RAT-Bench: A Comprehensive Benchmark for Text Anonymization.

https://arxiv.org/abs/2602.12806
Agyemang et al. (2026). LLM-Redactor: An Empirical Evaluation of Eight Techniques for Privacy-Preserving LLM Requests.

https://arxiv.org/abs/2604.12064
Yu et al. (2026). MINIM: Privacy-Aware Minimal View for Agents via Trusted Local Sanitization.

https://arxiv.org/abs/2606.13949
Brynjólfsson et al. (2026). RedactionBench.

https://arxiv.org/abs/2606.18782
Hu, Yu, & Wang (2026). Reversibility-Verified De-identification for Cloud-Local LLM Inference (DR-SL).

https://arxiv.org/abs/2609.14883
Ruzzetti et al. (2026). MuPPET: Contextual Privacy of LLM Assistants in Multi-Party Conversations.

https://arxiv.org/abs/2606.23217
Tamber et al. (2026). PrivacyAlign: Contextual Privacy Alignment for LLM Agents.

https://arxiv.org/abs/2606.21710
Kossen et al. (2024). Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs.

https://arxiv.org/abs/2406.15927